

Registros de decisión arquitectónica (ADR)

Los siguientes registros documentan las decisiones estructurales más relevantes del rediseño. Las alternativas incluidas son opciones técnicamente viables que podían resolver el problema con diferencias de alcance, cohesión o flexibilidad; no se presentan como alternativas soluciones que simplemente reproducen el defecto identificado. En cada ADR se explica por qué se prefirió una opción sobre las demás, qué costo se acepta y qué principios SOLID sustentan la decisión.

ADR-01 — Separar Hacienda por capacidades de negocio y conservarla como propietaria del estado

Contexto y evidencia

H-01 evidencia que Hacienda ejecutaba administración de potreros, manejo de reses, ventas y vacunación; H-03 muestra además que construía objetos y publishers asociados a esas mismas áreas. Las responsabilidades cambian por políticas diferentes y ya existían servicios que podían recibir los casos de uso.

Decisión arquitectónica a resolver

La decisión era cuánto debía seguir haciendo Hacienda después de retirar la concentración de lógica, sin perder el estado compartido que representa a la hacienda.

Alternativas evaluadas

- **Alternativa A — Mantener Hacienda como orquestador y extraer solo colaboradores especializados.** Hacienda conservaría los métodos públicos de los casos de uso, pero delegaría internamente en clases pequeñas. La ventaja es una migración menos invasiva y una fachada única para los consumidores. El costo es que Hacienda seguiría siendo el punto que cambia cuando se incorpora o reorganiza un caso de uso y continuaría conociendo demasiadas capacidades.
- **Alternativa B — Crear un único HaciendaService como capa de aplicación y mover allí toda la lógica retirada.** Reduce la lógica dentro de la entidad Hacienda y mantiene una entrada centralizada. Sin embargo, el nuevo servicio heredaría casi las mismas razones de cambio: potreros, reses, ventas y vacunación volverían a concentrarse en una sola frontera, solo que con otro nombre.
- **Alternativa C — Separar por capacidad de negocio y dejar Hacienda como propietaria del estado compartido.** PotreroService, ResService, VentaService y las fronteras de vacunación reciben los casos de uso correspondientes; Hacienda conserva las colecciones y expone capacidades estrechas cuando un consumidor necesita leer o registrar estado.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque la separación sigue los ejes reales de cambio del negocio y no solo la distribución física del código. Permite que una modificación sanitaria no obligue a abrir la lógica comercial y que una modificación de potreros no afecte vacunación. También aprovecha fronteras ya existentes, evita crear un servicio central sustituto de Hacienda y conserva una única fuente de verdad para el estado.

Costo o consecuencia negativa aceptada

La arquitectura contiene más objetos y requiere un composition root más explícito. Además, Hacienda sigue siendo un proveedor de varias capacidades de estado, por lo que debe garantizarse que Program comparta la misma instancia y no cree colecciones paralelas.

Principios involucrados

SRP y DIP, con apoyo de ISP en las interfaces de acceso al estado.

ADR-02 — Modelar cada tipo de potrero como una política coherente mediante IReglasTipoPotrero

Contexto y evidencia

H-04 y H-05 muestran que la incorporación de una res combina dos decisiones que siempre deben ser coherentes: qué edades acepta un tipo de potrero y qué subtipo de Res debe construirse. Además, el tipo textual del potrero es utilizado por UI y persistencia.

Decisión arquitectónica a resolver

La decisión era si separar la validación y la construcción en capacidades independientes o mantenerlas como una política única por categoría de potrero.

Alternativas evaluadas

- **Alternativa A — Separar IValidadorEdadPotrero e ICreadorRes.** Potrero recibiría dos abstracciones pequeñas: una validaría edad y otra construiría la Res. Es una separación limpia desde ISP, pero introduce la posibilidad de cablear combinaciones incoherentes, por ejemplo validar con la política de Ternero y construir un Novillo. Para impedirlo sería necesario agregar otra coordinación o configuración que mantenga ambas piezas sincronizadas.
- **Alternativa B — Hacer que cada subtipo de Res exponga su propia información de creación/edad y usar un registro de tipos.** Aprovecha el polimorfismo de la jerarquía Res, pero antes de crear la Res todavía no existe una instancia sobre la que preguntar por la política. Termina requiriendo objetos prototipo, metadatos estáticos o un segundo mecanismo de resolución, y mezcla la identidad de una entidad con la política del potrero que decide incorporarla.
- **Alternativa C — IReglasTipoPotrero con una implementación por categoría.** Cada ReglaTipoPotreroX declara TipoPotrero, valida la edad contextual de incorporación y construye el subtipo correspondiente. Las implementaciones se configuran en Program y Potrero solo consume la política recibida.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque validación y construcción forman un mismo eje de variación en este caso: ambas dependen de la categoría del potrero y deben evolucionar de forma consistente. Mantenerlas en una sola política evita estados de configuración incompatibles sin obligar a Potrero a conocer subtipos concretos. A la vez, la validez intrínseca de la edad de la Res se mantiene separada en IReglaEdadRes, por lo que IReglasTipoPotrero no absorbe toda la política de edad del dominio.

Costo o consecuencia negativa aceptada

La incorporación de una categoría exige crear una nueva implementación y registrarla en Program. La interfaz reúne dos operaciones relacionadas por coherencia de categoría, por lo que si en el futuro construcción y validación dejan de cambiar juntas deberá reevaluarse la frontera.

Principios involucrados

DIP y OCP; SRP se utiliza para mantener separados el eje “tipo de potrero” y el eje “edad intrínseca de la Res”.

ADR-03 — Mantener un único contrato Edad en Res y delegar la política a IReglaEdadRes

Contexto y evidencia

H-09, H-10 y H-11 muestran que Ternero, Cebon y Novillo redefinían Edad con condiciones propias. El cliente podía trabajar con una referencia Res y obtener una semántica distinta según el subtipo, comprometiendo la sustituibilidad.

Decisión arquitectónica a resolver

La decisión era dónde ubicar la variación legítima de los rangos de edad sin conservar múltiples contratos públicos para la misma propiedad.

Alternativas evaluadas

- **Alternativa A — Mantener Edad virtual en Res y sobrescribir únicamente una operación protegida ValidarEdad en cada subtipo.** Esta opción conserva un único setter público y reduce el riesgo respecto del AS-IS. Sin embargo, la regla queda ligada a la jerarquía y resulta difícil reutilizar exactamente el mismo rango desde Potrero o persistencia sin duplicarlo o hacer que otros componentes dependan de los subtipos.
- **Alternativa B — Validar la edad solo antes de construir la Res, fuera de la entidad.** Simplifica Res y evita inyectar una regla, pero debilita su invariante: una asignación posterior, una reconstrucción desde persistencia o un flujo distinto de Potrero podría introducir una edad inválida porque la entidad ya no protege su propio estado.
- **Alternativa C — Un único miembro Res.Edad apoyado en IReglaEdadRes.** Res mantiene la propiedad y protege la invariante; la política concreta de rango se compone. Las mismas reglas pueden reutilizarse desde ReglaTipoPotreroX y desde los flujos de reconstrucción.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque separa dos conceptos sin debilitar ninguno: Res conserva la responsabilidad de mantener un estado válido y las reglas concretas de rango permanecen sustituibles y reutilizables. A diferencia de los overrides, todos los consumidores observan exactamente el mismo contrato público Res.Edad. A diferencia de la validación externa, la invariante sigue protegida incluso fuera del flujo de incorporación a un potrero.

Costo o consecuencia negativa aceptada

Los constructores de Res y de sus subtipos necesitan recibir IReglaEdadRes, y Program debe componer correctamente cada subtipo con su regla. Esto aumenta el cableado y hace que la creación directa de una Res requiera conocer la política adecuada.

Principios involucrados

LSP, SRP y DIP.

ADR-04 — Usar IValidador<T> como contrato común y eliminar la jerarquía Validacion

Contexto y evidencia

H-12 y H-28 evidencian que IValidarInformacion obligaba a cada validador a declarar operaciones para Res, Potrero, Vacuna y Venta aunque solo pudiera ejecutar una; la superclase Validacion trasladaba ese contrato y los subtipos terminaban usando NotImplementedException.

Decisión arquitectónica a resolver

La decisión era cómo conservar la idea común de “validar” sin volver a prometer capacidades que cada implementación no puede cumplir.

Alternativas evaluadas

- **Alternativa A — Crear interfaces nominales separadas: IValidadorRes, IValidadorPotrero, IValidadorVacuna e IValidadorVenta.** Es una solución correcta de ISP y deja contratos muy explícitos. Su desventaja es repetir cuatro veces la misma forma contractual y dificultar la creación de componentes genéricos que solo necesitan “un validador de T”.
- **Alternativa B — Crear una clase abstracta genérica Validacion<T> con Validar(T).** También elimina operaciones incompatibles y permite compartir comportamiento futuro. El problema es que hoy no existe estado ni algoritmo común real; introducir una superclase solo para compartir una firma crea una relación de herencia sin beneficio concreto y obliga a justificar sustituibilidad donde una interfaz es suficiente.
- **Alternativa C — Definir IValidador<T> y hacer que cada validador concrete T.** Mantiene una única abstracción conceptual, evita métodos ajenos y permite que existan varias implementaciones para un mismo tipo sin ampliar el contrato.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque es la opción con menor compromiso estructural que expresa exactamente la capacidad compartida observada. Frente a cuatro interfaces nominales, evita duplicación contractual sin perder especificidad de tipo; frente a una clase abstracta genérica, no introduce una jerarquía cuando no hay implementación común que reutilizar. Además, facilita pruebas y futuros validadores adicionales para un mismo T.

Costo o consecuencia negativa aceptada

Se pierde un punto único de extensión por herencia. Si más adelante aparece lógica común significativa entre validadores, tendrá que extraerse por composición o justificarse una nueva abstracción; no se presupone desde ahora.

Principios involucrados

ISP y LSP, con apoyo de OCP para permitir nuevas implementaciones de validación.

ADR-05 — Mantener la serialización en PersistenciaService y abstraer la infraestructura mediante puertos específicos

Contexto y evidencia

H-13 y H-15 muestran que PersistenciaService mezclaba serialización, acceso físico a archivos, creación de proxies, validación y lectura de HttpContext. El problema combina responsabilidades técnicas diferentes, pero el sistema actual sigue utilizando los mismos formatos y operaciones de guardado/carga.

Decisión arquitectónica a resolver

La decisión era hasta dónde dividir persistencia: solo desacoplar infraestructura o reemplazar el servicio por repositorios/serializadores separados por entidad.

Alternativas evaluadas

- **Alternativa A — Crear un repositorio por agregado o entidad (IPotreroRepository, IResRepository, IVentaRepository, IVacunaRepository, etc.).** Produce una arquitectura muy explícita por dominio y facilitaría que cada agregado cambie de almacenamiento de forma independiente. Sin embargo, el código actual no evidencia operaciones de repositorio distintas por agregado: en gran parte prepara y recupera representaciones textuales. Adoptar repositorios implicaría redefinir contratos, ciclos de vida y semántica de persistencia más allá del hallazgo.
- **Alternativa B — Extraer también un serializador por tipo y dejar PersistenciaService únicamente como coordinador.** Reduce aún más su tamaño y podría favorecer cambios de formato independientes. El costo es multiplicar clases sin evidencia de que los formatos de Potrero, Res, Venta, Vacuna y Usuario tengan ciclos de cambio realmente independientes en esta fase.
- **Alternativa C — Conservar en PersistenciaService la preparación/interpretación de datos y abstraer almacenamiento, proveedor de validadores y resultado de validación.**
IAlmacenamientoHacienda oculta File/Directory/Path; IValidadoresGuardado oculta Castle/proxies; IResultadoValidacion oculta HttpContext. PersistenciaService mantiene el conocimiento lógico de los formatos actuales.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque resuelve exactamente los ejes de cambio demostrados por H-13/H-15 sin convertir la corrección en una sustitución completa del modelo de persistencia. El medio físico puede cambiar sin alterar la preparación de datos; el mecanismo de validación puede cambiar sin tocar serialización; y HttpContext queda fuera del módulo de alto nivel. Las alternativas A y B ofrecen mayor fragmentación, pero requieren asumir independencia de evolución que el sistema actual no demuestra.

Costo o consecuencia negativa aceptada

PersistenciaService sigue siendo un punto central para los formatos de serialización y deberá modificarse si cambia la representación lógica de una entidad. Se acepta ese acoplamiento porque corresponde a una responsabilidad todavía común y observable en el sistema actual.

Principios involucrados

SRP y DIP, con apoyo de ISP en los contratos de infraestructura.

ADR-06 — Separar el módulo de usuarios por responsabilidad manteniendo un único dueño del estado compartido

Contexto y evidencia

H-07 y H-20 muestran que Autenticacion y UsuarioService concentraban creación, consultas, autenticación, autorización, estadísticas, Claims e inicialización. H-08 agrega que las políticas de autorización cambian por rol. Todas esas operaciones necesitan observar el mismo conjunto de usuarios.

Decisión arquitectónica a resolver

La decisión era cómo dividir responsabilidades sin duplicar la colección ni introducir una interfaz “IUsuarios” que vuelva a reunir capacidades distintas.

Alternativas evaluadas

- **Alternativa A — Mantener un único servicio público de usuarios y componer internamente componentes privados especializados.** Los consumidores conservarían una dependencia simple y el servicio podría delegar. Sin embargo, el contrato seguiría exponiendo creación, consulta, autenticación, estadísticas y seguridad a clientes que solo necesitan una parte; cada ampliación obligaría a modificar la fachada y su interfaz, manteniendo un punto de cambio amplio.
- **Alternativa B — Separar servicios, pero entregar a cada uno una List<Usuario> compartida directamente.** Es simple y mantiene una única colección si Program comparte la misma lista. El problema es que todos los consumidores reciben capacidad de mutación total, el dueño del estado queda implícito y resulta más fácil introducir modificaciones accidentales o reemplazar la lista en un componente.
- **Alternativa C — Introducir un dueño explícito del estado y servicios/capacidades estrechos por responsabilidad.** RegistroUsuarios/UsuariosRegistradosHacienda mantiene la única colección; creación, consulta, autenticación, autorización, estadísticas e inicio de sesión consumen solo las capacidades necesarias. Las reglas de autorización se aíslan por rol.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque combina cohesión con control del estado compartido. A diferencia de una fachada única, los consumidores no dependen de operaciones que no usan. A diferencia de una List<Usuario> inyectada, la colección tiene un propietario explícito y puede exponerse en modo lectura o escritura según la necesidad. Esta decisión también permite que la autorización evolucione por nuevas reglas de rol sin reabrir autenticación o creación.

Costo o consecuencia negativa aceptada

Se incrementa el número de objetos y la configuración de dependencias. La corrección depende de que Program mantenga una única instancia del estado; un error de ciclo de vida podría generar registros divergentes.

Principios involucrados

SRP, OCP, ISP y DIP.

ADR-07 — Distribuir VacunaService en fronteras de caso de uso y mantener la variación de tipos fuera de la coordinación central

Contexto y evidencia

H-21, H-22 y H-23 muestran que VacunaService llegó a concentrar aplicación, creación, inventario, consultas, estadísticas, notificaciones y persistencia, además de conocer Bacteriana y Viva. H-29 evidencia que la presentación también seleccionaba variantes de creación.

Decisión arquitectónica a resolver

La decisión era si conservar un único servicio de vacunación como fachada/orquestador o exponer directamente capacidades especializadas a los consumidores.

Alternativas evaluadas

- **Alternativa A — Conservar VacunaService como fachada del módulo e inyectarle creadores, inventario, consulta, estadísticas, notificación y registro.** Es una opción viable: el servicio dejaría de conocer concretos y ofrecería una entrada única. La desventaja es que su interfaz seguiría creciendo con cada nuevo caso de uso y consumidores como el controlador dependerían de operaciones que no necesitan. Además, cambios en consultas o reportes continuarían obligando a modificar la fachada aunque no afecten la aplicación sanitaria.
- **Alternativa B — Separar únicamente la creación de Bacteriana/Viva y conservar aplicación, consulta, estadísticas e inventario dentro de VacunaService.** Ataca el eje OCP de tipos de vacuna con menos clases, pero deja sin resolver la concentración SRP documentada en H-23: reporte, consulta, mensajes y modificación del inventario continuarían compartiendo una misma razón de despliegue y prueba.
- **Alternativa C — Crear fronteras independientes: AplicacionVacuna, InventarioVacunas, ConsultaVacunas, EstadisticasVacunas, NotificacionVacunacion, RegistroVacunacion y creaciones específicas.** Cada consumidor recibe solo la capacidad requerida. Las entidades Bacteriana/Viva aportan su clasificación polimórfica y las clases de creación contienen los parámetros particulares de cada tipo.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque resuelve simultáneamente la concentración de responsabilidades y el eje de extensión por tipo sin crear un nuevo “servicio universal”. La aplicación sanitaria conserva la coordinación del caso de uso, pero consultas, estadísticas, mensajes e inventario pueden evolucionar independientemente. La alternativa A habría sido aceptable como fachada delgada, pero mantendría un contrato amplio y un punto central de modificación; la alternativa B no atendía todos los síntomas de H-23.

Costo o consecuencia negativa aceptada

El flujo de vacunación queda distribuido entre más colaboradores y requiere mayor esfuerzo para comprender la secuencia completa y para configurar Program. Las pruebas integrales necesitan ensamblar varias capacidades, aunque las pruebas unitarias se vuelven más focalizadas.

Principios involucrados

SRP, OCP, ISP y DIP.

ADR-08 — Mantener un único propietario del estado y distinguir explícitamente proveedor de consumidor en las interfaces

Contexto y evidencia

Durante H-16 se detectó una propuesta incorrecta en la que clases que trabajaban con los mismos potrereros podían interpretarse como proveedoras del mismo contrato. La revisión mostró que Hacienda es quien posee el estado; ConsultaPotrereros lo consulta y a su vez proporciona una capacidad de consulta; EstadisticasPotrereros consume esa consulta. H-24 aplica el mismo criterio al estado de ventas.

Decisión arquitectónica a resolver

La decisión era si extraer el estado a nuevos repositorios/almacenes o mantener la propiedad actual en Hacienda exponiendo capacidades de lectura y registro a los consumidores.

Alternativas evaluadas

- **Alternativa A — Extraer Potrereros y Ventas a objetos de estado independientes (por ejemplo, EstadoPotrereros y EstadoVentas).** Hacienda dejaría de poseer colecciones y los nuevos objetos serían proveedores claros. La opción reduce responsabilidades de estado en Hacienda, pero modifica el modelo de dominio y la propiedad de agregados más allá de lo que señalan H-16/H-24, con riesgo de introducir copias o cambiar la identidad de las colecciones existentes.
- **Alternativa B — Mantener Hacienda como propietaria y entregar la clase concreta a servicios de consulta y operación.** Preserva el modelo con muy pocos cambios y garantiza una sola fuente de verdad. Sin embargo, los consumidores quedan acoplados a todas las capacidades de Hacienda y cualquier cambio estructural en esa clase se propaga a servicios que solo necesitan leer o agregar un elemento.
- **Alternativa C — Mantener Hacienda como propietaria, pero exponer capacidades estrechas de estado y registro.** Hacienda realiza interfaces como IEstadoPotrereros/IRegistroPotrero e IEstadoVentas/IRegistroVenta. ConsultaPotrereros consume el estado y realiza IConsultaPotrereros; otros servicios consumen IConsultaPotrereros sin conocer Hacienda.

Decisión tomada y por qué se prefirió

Se eligió la alternativa C porque preserva la fuente de verdad y la identidad del estado del sistema original, pero evita que esa propiedad obligue a todos los consumidores a depender de Hacienda completa. La alternativa A ofrecía una separación más radical, pero no estaba motivada por un defecto de propiedad del estado; la alternativa B era más simple, pero mantenía el acoplamiento que H-16 y H-24 pretendían reducir. Esta decisión fija además una regla de diseño: una interfaz la realiza quien proporciona la capacidad, no quien simplemente la utiliza.

Costo o consecuencia negativa aceptada

Hacienda continúa implementando varios contratos de estado y registro, por lo que sigue teniendo un papel estructural importante. Program debe compartir la misma instancia para todas esas interfaces; la abstracción reduce acoplamiento, pero no elimina la necesidad de coordinar correctamente el ciclo de vida del estado.

Principios involucrados

DIP e ISP, con apoyo de SRP para separar propiedad del estado, consulta y comportamiento de aplicación.